

Unit 1.2

Node.js Fundamentals

The runtime, the event loop, modules and the project

02-eventloop-order.js

```
console.log('1');
setTimeout(() =>
  console.log('4'), 0);
Promise.resolve()
  .then(() =>
    console.log('3'));
console.log('2');

// why that order?
```

01

The runtime

What Node.js actually is, and what it is not

Node.js is three things in a trench coat

- V8 — Chrome's JavaScript engine, compiling and running your code. The language itself comes from here.
- libuv — a C library that provides the event loop and a thread pool, and wraps the operating system's asynchronous I/O.
- The standard library — the node: modules: file system, network, crypto, streams, child processes, and now SQLite and a test runner.

Node.js is not

- a programming language
- a web framework
- multi-threaded by default
- only for web servers

The consequence Everything you already know about JavaScript still applies. What changes is the environment around it: no DOM, but a file system, sockets and processes.

The browser and the server

In the browser

- Globals: window, document, localStorage, alert.
- Sandboxed: no access to the file system.
- One user, one tab, one session.
- You ship the code to the user, so size matters.

In Node.js

- Globals: process, Buffer, __dirname, require.
- Full access to files, sockets and processes.
- One process serving many users at once.
- The code stays on your server, so size matters less.

Shared by both: the language, the standard objects, promises, async/await, and fetch — which Node.js has had built in since v18.

code `fundamentals/00-hello.js`

02

The event loop

One thread, many requests — and the one rule you must not break

Single-threaded, non-blocking

Two statements that sound contradictory and are not.

- Single-threaded means your JavaScript runs on one thread. Two lines of your code never execute at the same time, so there are no data races and no locks.
- Non-blocking means that when your code asks for something slow — a file, a query, a request — it does not wait. It registers a callback and returns control.
- libuv does the waiting, using the operating system's own notification mechanisms (epoll, kqueue, IOCP) and a small thread pool for the operations that need it.
- When the result is ready, your callback is queued. The event loop runs it as soon as the thread is free.

So the rule is Never keep the thread busy. Waiting is free; computing is not. One slow loop and every user of that process is stuck behind it.

The phases of one turn

Every iteration of the loop walks these phases in order, draining one queue at a time.

- 1 **timers** callbacks scheduled by `setTimeout` and `setInterval` whose time has come
- 2 **pending callbacks** a few system operations deferred from the previous turn
- 3 **idle, prepare** internal to libuv; you never see this
- 4 **poll** where the loop spends its life: it asks the OS for finished I/O and runs those callbacks
- 5 **check** callbacks registered with `setImmediate`
- 6 **close callbacks** 'close' events, e.g. a socket being torn down

Microtasks are different `process.nextTick` and promise callbacks are not a phase. They are drained after every single callback, which is why they always run before the next timer.

Predict the output

Write down your answer before we run it. Most people get two of the six wrong.

```
fundamentals/02-eventloop-order.js
```

```
console.log('1 - sync start');

setTimeout(() => console.log(
  'timers - setTimeout 0'), 0);
setImmediate(() => console.log(
  'check - setImmediate'));

Promise.resolve().then(() => console.log(
  '4 - promise'));
process.nextTick(() => console.log(
  '3 - nextTick'));

console.log('2 - sync end');
```

The answer

- 1 and 2 first: synchronous code always finishes.
- Then 3, then 4: nextTick has priority over promises.
- Then the two timers — in either order.

Yes, really: from the main module, `setTimeout(0)` against `setImmediate` is not deterministic. Inside an I/O callback it is — `setImmediate` always wins.

Blocking the loop

What it looks like

- A 100 ms timer fires 2.5 seconds late, because a loop kept the thread busy.
- In a server: every request queues behind the slow one. Throughput collapses and health checks start failing.
- The usual culprits: huge JSON.parse, synchronous fs calls, crypto, image processing, regexes that backtrack.

What to do about it

- node:worker_threads — move the computation to another thread.
- child_process — or to another process entirely.
- Split the work into chunks and yield with setImmediate between them.
- Or simply do not do it here: a queue and a separate worker service.

Worth remembering Node.js is excellent at I/O-bound work and mediocre at CPU-bound work. Almost everything an API does is I/O-bound, which is why this model fits.

code `fundamentals/03-blocking.js`

03

Modules

CommonJS, ES modules, and how Node.js decides which one you meant

Two module systems

● CommonJS (.cjs)

- Node.js's original system: `require()` and `module.exports`.
- Synchronous and dynamic — you can `require()` inside an `if`.
- Gives you `__dirname` and `__filename`.
- Still the default for a `.js` file with no `"type"` in `package.json`.

● ES modules (.mjs)

- The language standard: `import` and `export`.
- Static — imports are resolved before the code runs.
- `import.meta.dirname` instead of `__dirname`; top-level `await` allowed.
- Active when `package.json` says `"type": "module"`.

Which one for this course? ES modules for anything new. You still have to read CommonJS, because a large part of the ecosystem is written in it.

code [fundamentals/modules-cjs/](#) · [fundamentals/modules-esm/](#)

The same module, twice

Exporting two functions and consuming them, in each system.

CommonJS – greeter.js / main.js

```
// greeter.js
function greet(name) {
  return `Hello, ${name}!`;
}
module.exports = { greet };

// main.js
const { greet } = require('./greeter');
const path = require('node:path');
console.log(greet('AT'), __dirname);
```

ESM – greeter.mjs / main.mjs

```
// greeter.mjs
export function greet(name) {
  return `Hello, ${name}!`;
}

// main.mjs
import { greet } from './greeter.mjs';
import path from 'node:path';
console.log(greet('AT'),
  import.meta.dirname);
```

Two habits worth having Prefix every built-in with node: — it makes the import unambiguous. And in ESM write the file extension: './greeter.mjs', not './greeter'.

How a specifier is resolved

Three kinds of import, three different rules.

node:fs

A built-in. Resolved immediately, never touches the disk. The node: prefix guarantees nobody can shadow it with a package of the same name.

./utils.mjs

A relative path. Resolved against the importing file. In ESM the extension is required; in CommonJS it is guessed.

express

A bare specifier. Node.js walks up looking for node_modules/express in every parent folder until it finds one or reaches the root.

This is why a missing dependency says "Cannot find module 'express'" and not "file not found": Node.js searched every node_modules above your file and found nothing.

04

npm and the project

package.json, semantic versioning, the lock file and scripts

npm is three things

The name is overloaded, which is a common source of confusion.

1

A registry

A public archive of over three million packages at npmjs.com. Anyone can publish, which is both its strength and the reason to look before you install.

2

A command-line tool

The npm command, bundled with Node.js. It installs, updates and removes dependencies, and runs the scripts you define. Yarn and pnpm are alternatives to this part.

3

A manifest format

package.json, the file that describes your project: its name, its entry point, its dependencies and its scripts. Everything else is derived from it.

Note Swapping npm for yarn changes the tool, not the registry or the manifest. The same package.json works with all of them.

package.json, field by field

The file npm init creates, and what each line is actually for.

package.json

```
{
  "name": "task-manager",
  "version": "1.0.0",
  "type": "module",
  "main": "src/index.js",
  "engines": { "node": ">=24" },
  "scripts": {
    "start": "node src/index.js",
    "dev": "node --watch src/index.js",
    "test": "node --test"
  },
  "dependencies": { "express": "^5.1.0" },
  "devDependencies": { "eslint": "^9.0.0" }
}
```

- type decides CommonJS or ESM for every .js file in the project.
- engines documents which Node.js you support — and warns whoever installs it.
- dependencies are needed to run; devDependencies only to develop.
- scripts are the project's public commands. Run them with npm run <name>.

Versions and the lock file

`^5.1.0` does not mean `5.1.0`. Knowing the difference is what keeps a project reproducible.

Semantic versioning

- MAJOR.MINOR.PATCH — `5.1.0`
- `^5.1.0` accepts `5.x.x`, not `6.0.0`
- `~5.1.0` accepts `5.1.x` only
- `5.1.0` exactly, nothing else

package-lock.json

- Records the exact version installed, for every package and every sub-dependency.
- Commit it. It is what makes your machine and mine install the same tree.
- `npm ci` installs from it, ignoring the ranges. Use it in CI.

The failure this prevents "It works on my machine" is usually a minor release that shipped a bug after your last install. Without a lock file, nobody can reproduce your tree.

Yarn writes `yarn.lock` instead. Same purpose, different format — never commit both.

A project from scratch

Five commands. This is the skeleton every lab session starts from.

1

Create and initialise

`mkdir task-manager && cd task-manager`, then `npm init -y`. You now have a `package.json`.

2

Choose your module system

Add `"type": "module"` so `import` works in `.js` files. Do it before writing code, not after.

3

Add dependencies

`npm install express` adds it to dependencies; `npm install -D eslint` adds it to devDependencies. Both update the lock file.

4

Define your scripts

A `"dev"` script with `node --watch` and a `"start"` script with plain `node`. Everyone who clones the repo now knows how to run it.

5

Ignore what is generated

A `.gitignore` with `node_modules` and `.env`. Never commit either: one is reinstallable, the other is secret.

Tools you no longer need to install

Node.js has absorbed several of the packages older tutorials tell you to add.

```
terminal
```

```
# restart on every save – instead of nodemon
```

```
$ node --watch src/index.js
```

```
# load variables from a file into process.env – instead of dotenv
```

```
$ node --env-file=.env src/index.js
```

```
# run the test files – instead of jest or mocha
```

```
$ node --test
```

```
# run a TypeScript file directly – instead of ts-node
```

```
$ node src/index.ts          # types are stripped, not checked
```

Talking to the outside world

process is the global object that connects your program to the shell that started it.

```
fundamentals/01-process.js

// node 01-process.js Alice 2
const [name, times = '1'] =
  process.argv.slice(2);

if (!name) {
  console.error('usage: node 01-process.js <name>');
  process.exit(1);
}

const greeting = process.env.GREETING ?? 'Hello';
console.log(`${greeting}, ${name}!`);
```

- argv[0] is the node binary and argv[1] this script — your arguments start at index 2.
- env is where configuration belongs: never hard-code a URL or a password.
- Exit code 0 means success. Anything else means failure, and that is what CI reads.
- stdout is for output, stderr for diagnostics. Keep them apart.

Coming in Unit 2 An HTTP server reads its port from process.env.PORT for exactly this reason: the same build must run on your laptop and on a server without editing a line.

KEY TAKEAWAYS

What to keep from this session

Node.js is V8 plus libuv plus a standard library.

The language is the one you already know; the environment around it is what changed.

One thread, and microtasks before the next phase.

Synchronous code, then nextTick, then promises, then timers. Never block the thread.

ES modules for new code, CommonJS to read.

"type": "module" decides it, and the node: prefix makes every built-in unambiguous.

package.json is the project; the lock file is the guarantee.

Commit the lock file, keep secrets in the environment, and install nothing globally.